

Requisitos de Migração e Integração

Este documento detalha os requisitos técnicos que o aplicativo_trilha (Flutter) exige do ambiente de produção (Servidor MAPL/PELMS, Broker de Campo e CLPIoTs) para funcionar corretamente.

1. Requisitos do Servidor de API (Backend MAPL/PELMS)

Nosso aplicativo foi construído contra um servidor `api_server.py` (Flask) que simula a API de produção. Para a migração ("virada de chave"), o servidor de produção do MAPL deve expor os seguintes endpoints HTTP:

- Ação de Migração (João/Lucas): Alteraremos a variável `baseUrl` no arquivo `lib/services/api_service.dart` para apontar para o IP/domínio do servidor de produção.

Endpoints Obrigatórios: O servidor de API do MAPL/PELMS deve possuir

1. POST `/api/login`
 - Função: Autentica um usuário.
 - Recebe (JSON): `{"email": "...", "senha": "..."}`
 - Retorna (JSON): `{"id_usuario": 1, "nome": "...", "tipo_perfil": 1, "url_foto_perfil": "..."}`
2. POST `/api/register`
 - Função: Cria um novo usuário (Trilheiro ou Funcionário).
 - Recebe: `multipart/form-data` (NÃO JSON).
 - Campos de Texto: `nome`, `email`, `senha`, `tipo_perfil` (1=Trilheiro, 2=Guia, 3=Operador), `telefone` (opcional), `idade` (opcional), `sexo` (opcional), `admin_code` (opcional).
 - Arquivo: `foto_perfil` (o arquivo da imagem).
 - Lógica de Negócio (Requisito): O servidor deve validar o `admin_code` se `tipo_perfil > 1`.
 - Retorna (JSON): Os mesmos dados do endpoint `/api/login` (para login automático).
3. POST `/api/trilha/iniciar`
 - Função: Registra o início de uma nova sessão de trilha.
 - Recebe (JSON): `{"id_usuario_lider": 1, "nome_trilha": "...", "dificuldade": "Média", "tipo": "Grupo", ...}`
 - Retorna (JSON): `{"id_trilha_ativa": 42}`
4. GET `/api/trilha/detalhes/<id>`
 - Função: Busca os dados para o Drawer (menu lateral) da trilha.

- Requisito Crítico: A data iniciada_em DEVE ser formatada como ISO 8601 (ex: "2025-11-16T14:30:00") para que o Dart possa interpretá-la e o timer funcione.
 - Retorna (JSON): {"trilha_info": {"nome_trilha": "...", "iniciada_em_iso": "..."}, "participantes": [{"nome": "...", "url_foto_perfil": "..."}]}
5. POST /api/trilha/finalizar/<id>
- Função: Marca uma trilha como concluída.
 - Recebe: ID da trilha na URL.
 - Retorna (JSON): {"mensagem": "Trilha finalizada"}
6. GET /api/clima?lat=...&lon=...
- Função: Busca dados de clima para a localização do usuário (via OpenWeatherMap ou similar).
 - Retorna (JSON): {"temp": 25.0, "descricao": "Céu limpo", "icone": "01d"}
7. GET /api/eventos/tag/<id_tag>
- Função: (Fluxo do Guia - Fase 4) Retorna quem passou por uma tag.
 - Retorna (JSON): {"passaram_por_aqui": [{"nome_usuario": "...", "timestamp_leitura": "..."}]}

Mecanismo de Boas-Vindas por E-mail

Funcionalidade de envio automático de e-mail de boas-vindas, disparada durante o cadastro de um novo usuário.

1. Objetivo da Funcionalidade

O objetivo deste mecanismo é fornecer feedback imediato e profissional ao usuário no momento em que ele cria uma conta (Trilheiro, Guia ou Operador). Assim que o cadastro no api_server.py é bem-sucedido, o servidor envia ativamente um e-mail de "Boas-vindas" para o endereço fornecido.

2. Componentes e Arquitetura

O envio de e-mail não é feito pelo aplicativo Flutter. Ele é uma funcionalidade de Backend (Servidor).

1. Serviço: api_server.py (Nosso servidor Flask/Python).
2. Bibliotecas Python: smtplib, ssl, e email.mime (todas nativas do Python).
3. Provedor de E-mail: Utilizamos o servidor SMTP do Google (Gmail) como nosso provedor de envio.

4. Autenticação (A "Chave API"): Como você mencionou, não usamos a senha real da conta de e-mail. Usamos uma "Senha de App" (App Password) de 16 dígitos gerada pelo Google.

O Fluxo de Cadastro de Funcionários (Guias/Operadores)

Para controlar quem pode criar contas com privilégios elevados (Guia ou Operador de Base), implementamos um fluxo de cadastro que exige um "Código de Administrador" (uma senha mestra).

1. Como Funciona no Aplicativo (Frontend)

Construímos a lógica na tela `register_screen.dart` (Fase 5.5) para reagir dinamicamente ao tipo de conta que o usuário deseja criar:

1. Seleção de Perfil: O usuário acessa a tela de "Criar Nova Conta". Ele vê um campo `DropDownButtonFormField` (lista suspensa) chamado "Tipo de Conta".
2. Opção "Trilheiro": Se o usuário selecionar "Trilheiro" (que é o `tipo_perfil = 1`), o formulário permanece simples (nome, e-mail, senha, etc.).
3. Opção "Funcionário" (Guia ou Operador): Se o usuário selecionar "Guia" (`tipo_perfil = 2`) ou "Operador de Base" (`tipo_perfil = 3`), um novo campo de texto aparece dinamicamente no formulário, chamado "Código de Administrador".
4. Validação (App): O formulário exige que este campo seja preenchido se o `tipo_perfil` for 2 ou 3.

2. Como Funciona no Servidor (Backend)

O aplicativo envia os dados do formulário (incluindo o `tipo_perfil` e o `admin_code`) para o endpoint `POST /api/register`.

1. Código Mestre: No nosso `api_server.py`, nós definimos uma constante (uma senha mestra) no lado do servidor:
 - `ADMIN_REGISTER_CODE = "MAPL_ADMIN_2025"`
2. Lógica de Validação: Quando o endpoint `/api/register` é chamado, o servidor executa a seguinte lógica (em Python):

Python

```
tipo_perfil = data.get('tipo_perfil', 1)
admin_code = data.get('admin_code')
```

```
# Se o usuário quer ser Guia (2) ou Operador (3)
```

```
if tipo_perfil > 1:
```

```
# Ele DEVE fornecer o código correto
```

```
if admin_code != ADMIN_REGISTER_CODE:
```

```
    # Se o código estiver errado, o servidor retorna 403 (Proibido)
```

```
    return jsonify({"erro": "Código de administrador inválido"}), 403
```

```
# Se o código estiver correto (ou se for um Trilheiro), o cadastro continua...
```

```
...
```

3. Resultado: É impossível para um usuário comum criar uma conta de Guia ou Operador sem saber este código secreto que só existe no servidor.

3. Necessário ser implementado no servidor

Para que este fluxo funcione na produção, a equipe de backend deve:

1. Implementar a Lógica: O endpoint POST `/api/register` do servidor de produção (PELMS) deve replicar a lógica de validação acima. Ele deve verificar o `tipo_perfil` e o `admin_code`.
2. Definir a Senha Mestra: A equipe deve definir qual será a "Senha Mestra" (`ADMIN_REGISTER_CODE`) real para o ambiente de produção. (O nosso "MAPL_ADMIN_2025" foi apenas um exemplo para o mock).
3. Comunicar a Senha: A equipe (ou o administrador do sistema) deve comunicar esta Senha Mestra aos gestores da trilha. Quando um novo Guia for contratado, o gestor deverá fornecer este código a ele para que ele possa criar sua conta no aplicativo.

3. Fluxo de Funcionamento Detalhado

O processo é o seguinte:

1. O Gatilho (Trigger):
 - O usuário preenche o formulário na tela RegisterScreen do aplicativo.
 - O app envia os dados (incluindo a foto) para o endpoint POST `/api/register` no nosso `api_server.py`.
2. A Lógica no Servidor (`api_server.py`):
 - O servidor recebe os dados.
 - Ele valida as informações (ex: Código de Admin, se for Guia).
 - Ele criptografa a senha (`generate_password_hash`).
 - Ele salva o novo usuário no banco de dados MySQL.

- Imediatamente após o `conexao.commit()` (confirmando que o usuário foi salvo), o script chama a nossa função interna:
`send_welcome_email(email, nome)`.
3. A Função `send_welcome_email(email, nome)`:
- Segurança: A função primeiro lê duas Variáveis de Ambiente do sistema operacional do servidor:
 1. EMAIL_USER (O e-mail do remetente, ex:
`app.trilha.mapl@gmail.com`)
 2. EMAIL_PASS (A "Senha de App" de 16 dígitos)
 - *Nota:* Isso segue as "Melhores Práticas" que definimos, de não colocar senhas no código.
 - Conexão: O script usa a biblioteca `ssl` e `smtpplib` para se conectar ao servidor do Google (`smtp.gmail.com`) na porta 465 (SMTPS).
 - Autenticação: Ele faz login no servidor SMTP usando o EMAIL_USER e a EMAIL_PASS. (Foi aqui que vimos o erro `BadCredentials` em nossos testes, quando as variáveis não estavam definidas).
 - Envio: Ele constrói um e-mail em formato HTML (usando `MIMEMultipart`) para saudar o novo usuário pelo nome (ex: "Olá, João Marcos!") e o envia.
 - Falha Silenciosa: Nós programamos o envio de e-mail para "falhar silenciosamente". Se o envio falhar (ex: `BadCredentials`), o servidor *não* cancela o cadastro do usuário. Ele apenas imprime o erro no log e retorna o sucesso 201 (Usuário Criado) para o aplicativo.

4. Requisitos de Migração

Para que esta funcionalidade funcione no servidor de produção, é necessário efetuar:

1. Firewall do Servidor: O servidor que hospedará o `api_server.py` deve ter permissão de saída (outbound) na porta TCP 465 (para que o Python possa se conectar ao `smtp.gmail.com`).
2. Conta de E-mail de Serviço: A equipe precisa criar ou designar uma conta de e-mail (preferencialmente Gmail/Google Workspace) para ser o remetente (*já foi criada e está presente na documentação*).
3. Gerar a "Senha de App" (A Chave API):
 - A "Verificação em Duas Etapas" deve estar ativa nesta conta de e-mail.
 - A equipe deve ir em "Conta Google" -> "Segurança" -> "Senhas de app".
 - Gerar uma nova senha, dando o nome "API Servidor Trilha".
 - Salvar a senha de 16 dígitos (ex: `abcd efgh ijkl mnop`).
(*já foi criada e está presente na documentação*).

4. Definir as Variáveis de Ambiente: No servidor de produção onde o `api_server.py` será executado, eles devem definir as variáveis de ambiente `EMAIL_USER` (com o e-mail) e `EMAIL_PASS` (com a senha de 16 dígitos) antes de iniciar o script.

2. Requisitos do Broker (Infraestrutura de Campo)

- Ação de Migração (João/Lucas): Alteraremos as variáveis `_broker` e `_port` no arquivo `lib/services/mqtt_service.dart`.
- Requisito: A equipe de campo deve fornecer o IP e a Porta do Broker MQTT Local (o Raspberry Pi na portaria).
- Requisito de Tópico: O Broker deve "ouvir" o tópico `trilha/eventos/passagem` (ou outro nome a ser definido). O nosso app publicará neste tópico.
- Formato dos Dados (MQTT): O app envia um JSON com este formato:

JSON

```
{  
  "id_usuario": 2,  
  "id_tag": 5,  
  "timestamp_leitura": "2025-11-16T18:00:00.123Z",  
  "direcao": "ida",  
  "latitude": -19.354978,  
  "longitude": -43.606314  
}
```

3. Requisitos de Hardware (Tags NFC/CLPIoT) - O Ponto Crítico

Esta é a parte mais importante, baseada em nossos testes de depuração com o HCE. Se estes requisitos não forem seguidos, o app "Trilheiro" rejeitará a tag (com o erro "Tag não suporta NDEF").

O hardware de campo (CLPIoT ou tag passiva) deve disparar perfeitamente uma Tag NDEF Tipo 4.

1. Protocolo AID (A "Saudação"):

- A tag *deve* responder ao comando APDU `SELECT AID` para o AID NDEF padrão: `D2760000850101`.
- *Log de Teste*: Nossos testes mostram que o app envia `00A4040007D276000085010100`. A tag deve responder `9000` (OK).

2. O Arquivo de Capacidade (CC File):

- Este foi o nosso principal ponto de falha. O app "Trilheiro" é *extremamente* rigoroso com este arquivo.
- Seleção: A tag deve responder ao SELECT FILE pelo ID E103.
- Formato de Leitura (Crucial): A tag deve suportar a leitura em duas etapas que vimos nos logs:
 1. O app primeiro lê 2 bytes (o NLEN, ex: 000D para 13 bytes).
 2. Depois, o app lê os 13 bytes restantes (o conteúdo) a partir do offset 2.
- Conteúdo (13 bytes): O conteúdo do CC File *deve* anunciar um arquivo NDEF gravável. Recomendamos a seguinte estrutura (que funcionou no nosso emulador):
 - 20: Versão (2.0)
 - 00FF: MLe (Tamanho máx. leitura)
 - 00FF: MLc (Tamanho máx. escrita)
 - 04 06: Par T-L (Tipo 4, 6 bytes de dados)
 - E104: File ID do arquivo NDEF
 - FFFE: Tamanho máx. do arquivo NDEF
 - 00: Acesso de Leitura (Permitido)
 - 00: Acesso de Escrita (Permitido)

3. O Arquivo NDEF (O Protocolo de Dados):

- Seleção: A tag deve responder ao SELECT FILE pelo ID E104 (ou o que for definido no CC File).
- **Requisito 3.1: O Registro Mestre (ID Lógico):**
 - A equipe de hardware deve pré-programar (provisionar) todas as tags com este registro.
 - O Registro NDEF 0 (o primeiro da lista) DEVE ser um Registro de Texto (Text Record).
 - O payload desse registro deve ser um JSON contendo o ID lógico da tag.
 - Exemplo para a Tag 5: {"logical_id": 5}
 - *Falha:* Se este registro faltar, o app "Trilheiro" (NfcInteractionDialog) mostrará "Tag inválida! Não é uma tag da trilha."
- **Requisito 3.2: O Protocolo de "Append" (Read-Add-Write):**

- O nosso app não faz um "append" simples. Ele reescreve a tag inteira (como uma transação).
- **Fluxo do Trilheiro:**
 1. O app (via `nfcService.readTagData`) lê a `NdefMessage` completa da tag (Registro 0: Mestre, Registros 1-N: Logs Antigos).
 2. O app (via `nfcService.writeToTag`) cria um *novo* Registro de Texto (ex: `{"id": "10", "ts": "..."}).`
 3. O app adiciona este novo registro à lista.
 4. O app envia o comando WRITE (como o 00D6 do HCE) para sobrescrever a `NdefMessage` inteira da tag com a nova lista (Reg 0 + Logs Antigos + Novo Log).
- **Requisito para o CLP IoT:** O hardware deve permitir esta operação de Leitura-Adição-Sobrescrita. A sua capacidade de memória deve ser suficiente para o Registro Mestre mais N logs de usuários (estimamos $N=250$).